

Software Requirements Specification

Logistics System

Ege Türker
Beste Kaya
Ata Türker

INF 439 Software Engineering

1.0. Introduction

1.1 Project Name & Description

The Logistics Management System is a desktop-based software application developed to manage and track shipment processes in a simple and structured manner. The system allows administrators to manage users and shipments, staff members to update delivery statuses during the distribution process, and public users to track shipments using a unique tracking number without authentication.

The main purpose of the system is to provide transparency, basic control, and ease of management for logistics operations. By centralizing shipment information and user roles in a single application, the system reduces manual tracking efforts and improves accessibility to shipment status information. The application is developed as a minimum viable product (MVP) and focuses on core logistics functionalities rather than advanced features.

1.2 Project Team Members and Roles

The Logistics Management System project is developed by a three-member team. Each team member contributed to different phases of the Software Development Life Cycle (SDLC), including requirements analysis, system design, and documentation.

- ***Ege Türker***

Responsible for preparing the project documentation and UI mockups. Took the lead in writing the Software Requirements Specification (SRS) and designing the overall structure of the report.

- ***Beste Kaya***

Responsible for the database design and user interface implementation. Designed the database schema and contributed to implementing the application's user interfaces based on the defined requirements.

- ***Ata Türker***

Responsible for the software implementation and coding tasks. Implemented the core functionalities of the system and integrated them with the database and user interface components.

All team members collaborated closely during the requirements analysis, system design, and decision-making processes. In addition, the implementation and coding tasks were carried out collaboratively, with team members supporting each other during development. Key architectural and design decisions were made jointly to ensure that the final system meets the project objectives and course requirement.

1.3 Problem Statement

Logistics and shipment tracking processes are often managed manually or through fragmented systems, making it difficult for organizations to track deliveries efficiently and provide accurate shipment information to customers. Lack of centralized control can lead to delayed updates, limited transparency, and increased operational complexity.

The problem addressed by this project is the absence of a simple, role-based logistics management system that allows different users to interact with shipment data according to their responsibilities. Administrators need a way to manage users and shipments, staff members require a controlled interface to update delivery statuses, and customers expect to track their shipments easily using a tracking number without creating an account.

The Logistics Management System aims to solve these issues by providing centralized desktop application that supports user management, shipment tracking, and delivery status updates through clearly defined roles and use cases.

1.4 Purpose

The purpose of this document is to present a detailed description of the Logistics Management System. This document explains the purpose and main functionalities of the system, including shipment tracking, user management, and delivery status updates. It describes how different user roles such as admin, staff, and public users interact with the system.

The document outlines the functional and non-functional requirements, system constraints, and expected system behavior in response to user actions. This Software Requirements Specification (SRS) is intended for stakeholders, instructors, and developers involved in the design and implementation of the system. It serves as a reference document throughout the development process.

2.0 Requirements Specifications

2.1 Functional Requirements

The functional requirements defines the behaviors and operations of the system. These are terms of our project to understand the infrastructure of the project.

Term	Definition
Admin	User with full system management privileges
Staff	User responsible for shipment delivery operations
Customer (Public User)	User who tracks shipments using a tracking number
Shipment	A package registered in the logistics system
Tracking Number	Unique code used to track a shipment

FR1 Login:

Description: The system shall authenticate registered users (Admin and Staff) and provide access to the application based on their specific roles.

Acceptance Criteria: The system must validate user credentials against the database upon login. The system must redirect users to their specific dashboard (Admin or Staff) based on their role.

FR2 Create New User:

Description: The system shall allow the Admin to register new users (other Admins or Staff) into the system.

Acceptance Criteria: The system must save the new user's information (username, password, role) to the database. The system must prevent the creation of a user with a duplicate username.

FR3 Update User:

Description: The system shall allow the Admin to modify the details of existing users.

Acceptance Criteria: Changes made to user information (e.g., password or role) must be immediately updated in the database. The Admin must be able to select a user from a list before editing their details.

FR4 Delete User:

Description: The system shall allow the Admin to permanently remove a user account from the system.

Acceptance Criteria: The deleted user must be removed from the database and must not be able to log in again. The system must show a confirmation prompt before the user is deleted to prevent accidental removal.

FR5 Create Shipment:

Description: The system shall allow authorized users to add new shipment records, including a unique tracking number.

Acceptance Criteria: The system must generate or accept a unique tracking number for each shipment. The shipment record must be saved with an initial status (e.g., "Out for Delivery", "Received" etc.).

FR6 Update Shipment Status:

Description: The system shall allow Admin and Staff to change the current status of a shipment (e.g., "Out for delivery", "Delivered" etc.).

Acceptance Criteria: * Users must be able to select the new status from a predefined list (Radio buttons and a text if it's necessary). The updated status must be visible to public users on the tracking screen immediately after the change is saved.

FR7 View Shipment:

Description: The system shall allow authorized users (Admin and Staff) to view a comprehensive list of all shipments stored in the database.

Acceptance Criteria: The system must display shipment details in a structured data grid view. The list must update in real-time or upon refreshing to show the most recent shipment data.

FR8 Track Shipment:

Description: The system shall allow public users (customers) to query the status of a specific shipment using its unique tracking number without needing to log in.

Acceptance Criteria: The system must provide a dedicated search field for the tracking number. If a valid number is entered, the system must display the current status and shipment details. If an invalid number is entered, the system must show a "Shipment not found" error message.

2.2 Non-Functional Requirements

NFR1 Usability:

Description: The system shall be intuitive and easy to use for users with basic computer knowledge (non-technical users).

Acceptance Criteria: A new user should be able to perform a "Shipment Tracking" or "Status Update" task without external documentation or help.

NFR2 Performance:

Description: The system must ensure quick response times for critical operations.

Acceptance Criteria: Tracking results and database queries must be processed and displayed on the screen within 2 seconds under normal load.

NFR3 Security:

Description: Access to administrative and operational functions must be restricted to authorized users only.

Acceptance Criteria: The system must prevent access to "Admin" and "Staff" panels if a valid session/login is not active. Password fields must mask the input.

NFR4 Reliability:

Description: The system shall maintain data integrity and prevent data loss during operations.

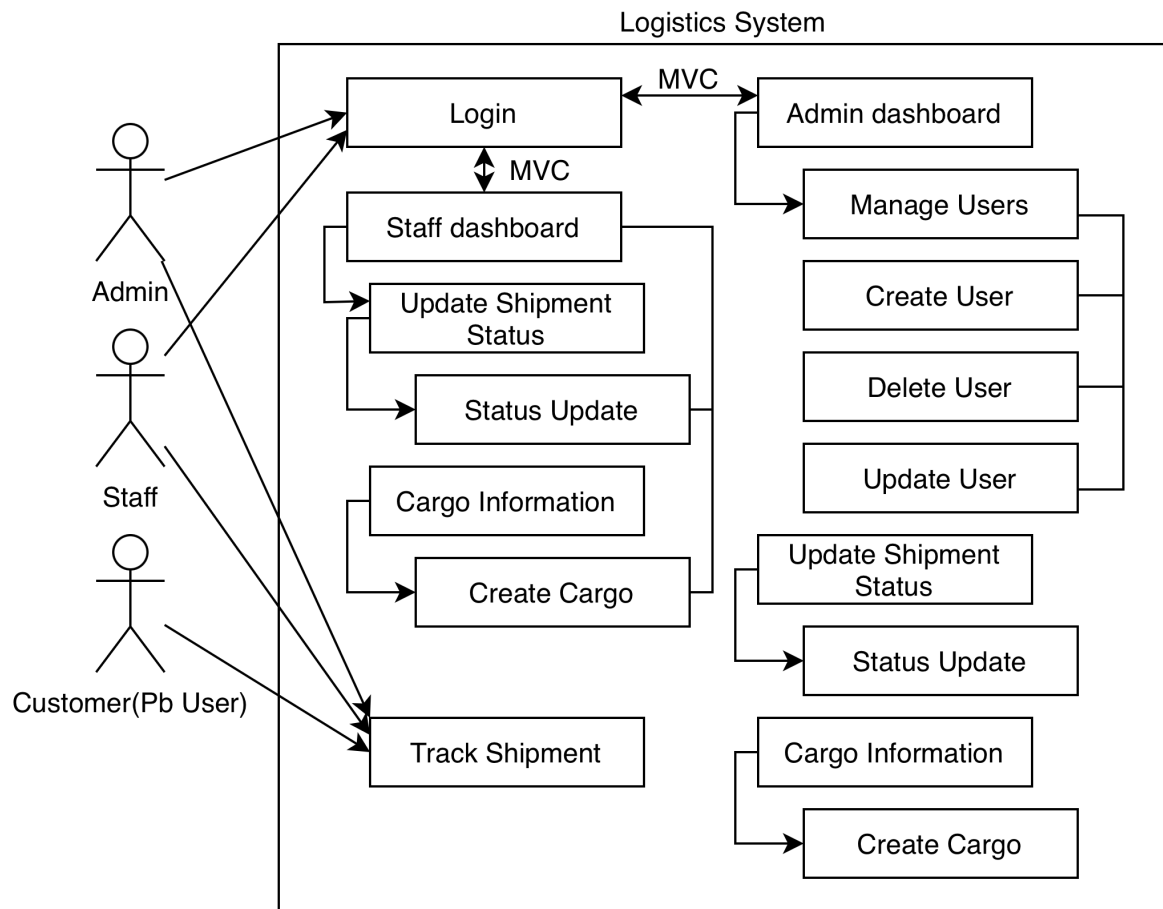
Acceptance Criteria: In case of an unexpected application crash, the system should not leave the database in a corrupted state; all committed transactions must be permanent.

NFR5 Maintainability

Description: The software architecture should be structured to support future updates and feature enhancements.

Acceptance Criteria: The source code must follow modular programming principles (e.g., separating UI logic from Database logic) so that new shipment statuses or user roles can be added without rewriting the core system.

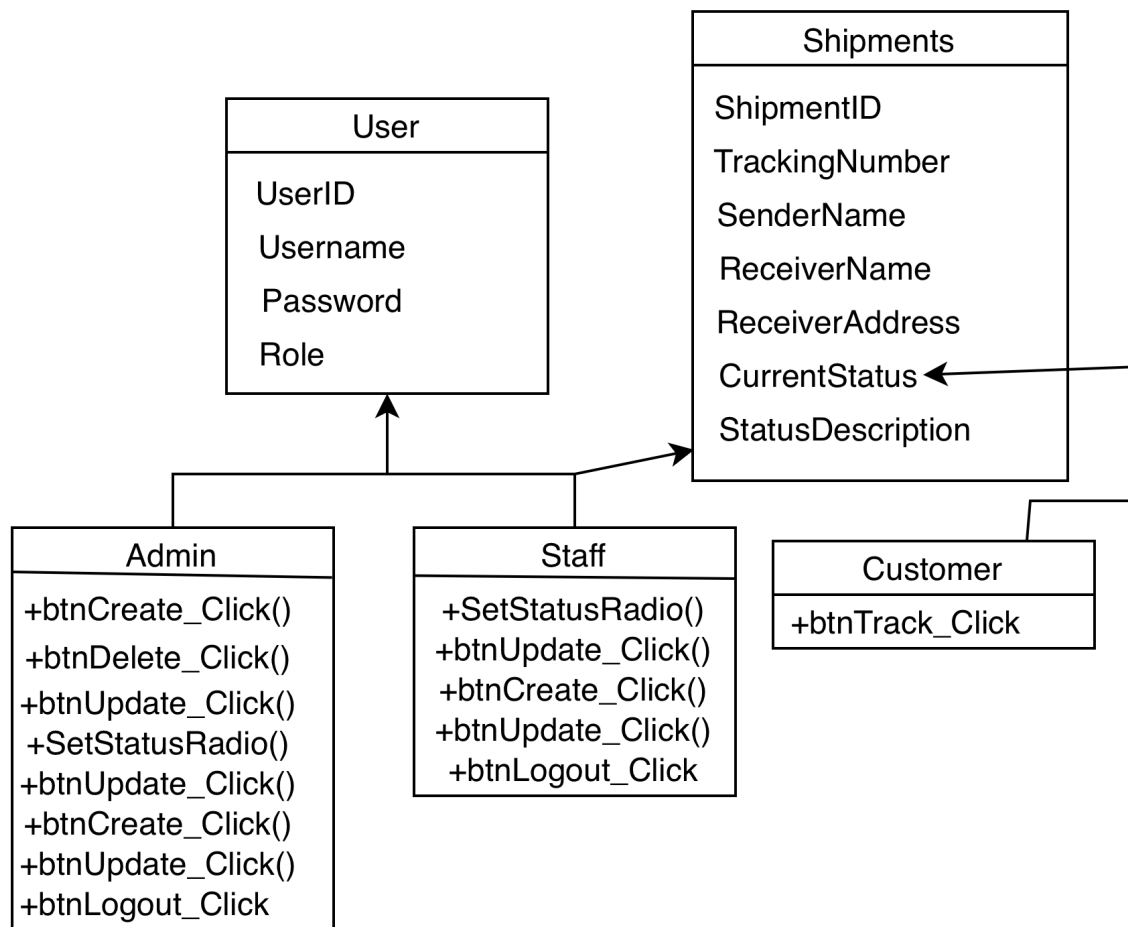
2.3 Use Case Diagram



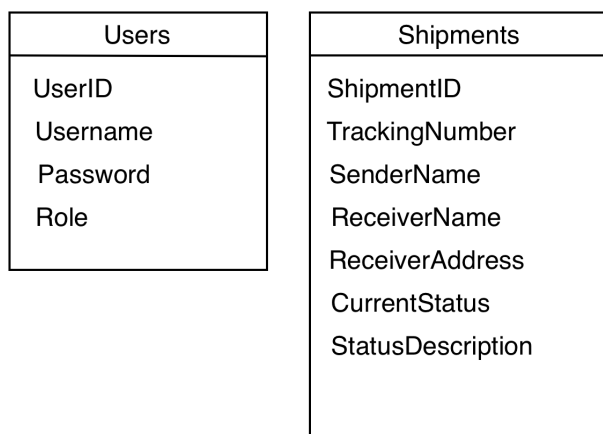
These diagrams are not generated with AI

3.0 System Design

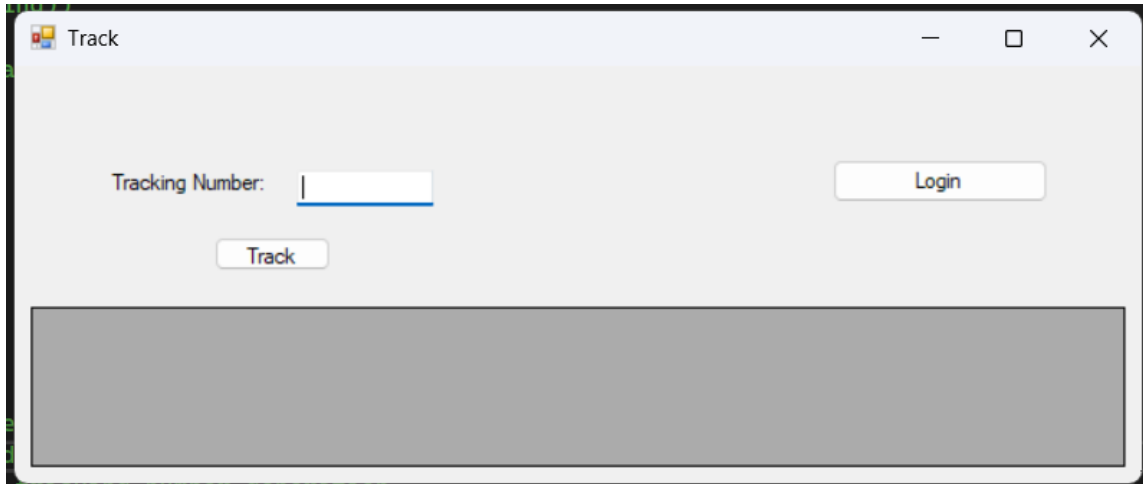
3.1 Class Diagram



3.2 Database Design(ER diagram)

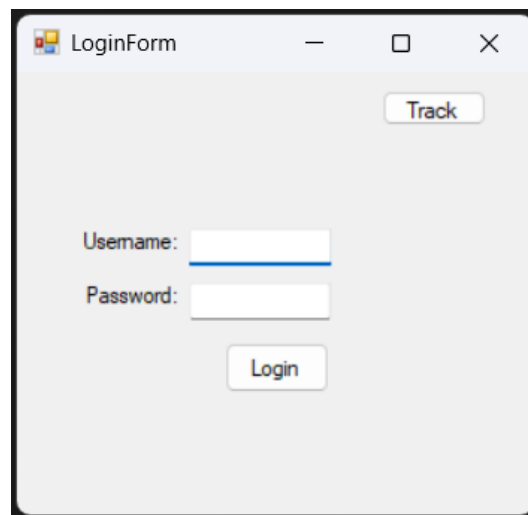


3.3 UI Mockups



The image shows a window titled "Track" with a light gray background. It features a "Tracking Number:" label followed by a text input field. Below the input field is a "Track" button. To the right of the input field is a "Login" button. At the bottom of the window is a large, empty gray rectangular area.

This is the primary form of the application. Since the number of Admin and Staff users is significantly lower than customers, the interface is specifically optimized for the Customer (Public User) tracking use case.



The image shows a window titled "LoginForm" with a light gray background. It features a "Track" button in the top right corner. Below it are labels for "Username:" and "Password:" followed by text input fields. At the bottom center is a "Login" button.

Login Form implemented with MVC architecture, this screen directs users to the appropriate Admin or Staff dashboard according to their specific role stored in the database.

AdminDashboard

Manage Users

Update Shipment Status

Cargo

Logout

Username :

Create

Password:

Update

Role:

Delete

	UserID	Username	Password	Role
▶	1	admin1	123	Admin
	2	staff1	123	Staff
	2002	test1	123	Staff
*				

Admin dashboard

StaffDashboard

Update Shipment

Cargo

Logout

Cargo List:

	ShipmentID	TrackingNumber	CurrentStatus	StatusDescription
▶	2	66185292	At Hub	Local distribution ce...
	3	68392858	Created	
*				

Status

Cancelled

Accepted

At Hub

In Transit

Out For Delivery

Delivered

Delivery Failed

Returned

Other

Update

Staff dashboard

4.0 AI and Prompt Engineering Log

Prompt 1

Task: Designing the database schema

My Prompt:

“I currently have two tables in my database: Users and Shipments. Do I need to define a relationship between these tables, or can they remain independent for an MVP logistics system?”

AI Output Summary:

The AI explained that a relationship between Users and Shipments is not mandatory if shipment records are not directly associated with a specific user. It suggested that relationships are only required when tracking ownership, assignment, or history.

My Refinement:

Based on the AI’s explanation, I decided not to create a relationship between the Users and Shipments tables. This decision aligns with the MVP scope of the project, where user roles are used only for authorization purposes and shipment data is managed independently.

Prompt 2

Task: Designing the Class Diagram

My Prompt:

“I need to create a class diagram for my project, but I’m not really sure how to do it or what it should look like. Can you help me with that?”

AI Output Summary:

The AI explained what a class diagram is and suggested creating simple classes that represent the main entities of the system, such as users and shipments.

My Refinement:

I followed the guidance and created a simple class diagram that focuses on the main data structures of the system, keeping it aligned with the MVP scope of the project. However, I initially found it a bit confusing because the class diagram looked very similar to the database structure.

Prompt 3

Task: Deciding how many forms are needed for the application

My Prompt:

“For a simple logistics management system, how many forms would you recommend creating? I want to keep the project simple but still functional.”

AI Output Summary:

The AI suggested a relatively large number of forms, including separate screens for different shipment actions and management tasks, which felt unnecessary for a simple MVP-level logistics system.

My Refinement:

We initially tried to follow the form structure suggested by the AI, but this approach

ended up complicating the project and making the system harder to manage. This experience helped us realize that leaving architectural decisions entirely to AI can sometimes increase complexity instead of reducing it. After recognizing this issue, we stepped back and redesigned the software architecture ourselves, focusing on simplicity and the actual needs of the MVP.

Prompt 4

Task: Retrieving data from the database

My Prompt:

“While working with the database, I needed help writing SELECT queries to retrieve user and shipment data. Can you help me with simple queries for fetching shipment status and basic user information?”

AI Output Summary:

The AI suggested various SELECT queries, including additional conditions and filtering options. Some of these suggestions were more detailed than necessary for the simple structure of the current database.

My Refinement:

I kept the queries very simple and used only basic SELECT statements that directly retrieve shipment status and user information from single tables, since the database does not include relational joins and the project follows an MVP approach.